Proyecto 15: Análisis de Accesos a Memoria y Rendimiento de Caché en Algoritmos de Ordenamiento

Richard Vasquez Genesis Cabello

12 de octubre de 2026

Resumen

El presente trabajo evalúa el impacto de la jerarquía de memoria en el rendimiento de los algoritmos de ordenamiento QuickSort y MergeSort mediante la simulación de una memoria caché de mapeo directo de 32 KB. Se analiza la variación de la tasa de aciertos (Hit Rate) y fallos (Misses) al modificar el tamaño de bloque desde 16 Bytes hasta 256 Bytes sobre un conjunto de datos de 10,000 elementos enteros.

1. El Problema

En la arquitectura de computadores moderna, la brecha de velocidad entre el procesador y la memoria RAM principal constituye uno de los principales cuellos de botella en el rendimiento de ejecución. A pesar de que algoritmos como QuickSort y MergeSort presentan una complejidad temporal promedio de $\mathcal{O}(N \log N)$, su desempeño práctico en hardware real depende críticamente de la **localidad de referencia** (espacial y temporal) con la que acceden a la memoria.

El problema radica en determinar cuantitativamente cómo afecta la variación del tamaño de bloque (línea de caché) a la tasa de aciertos (Hit Rate) de una memoria caché de mapeo directo de 32 KB al procesar arreglos contiguos de 10,000 enteros (40 KB de datos total). Específicamente, se busca responder:

1. ¿Cómo influye escalar el bloque desde 16 Bytes hasta 256 Bytes en la reducción de fallos de caché (Misses)?

2. ¿Qué algoritmo presenta un comportamiento más eficiente respecto al patrón de accesos a memoria según la arquitectura simulada?

2. Técnicas Computacionales

Para abordar el problema se diseñó e implementó un simulador de arquitectura en C++ que instrumentaliza las operaciones de lectura y escritura sobre los arreglos de datos.

2.1. Modelo de Memoria Caché de Mapeo Directo

Se modeló una estructura de caché con una capacidad fija de $C = 32KB = 32,768$ Bytes. En el esquema de mapeo directo, cada dirección de memoria física se mapea a una línea única mediante la siguiente descomposición de la dirección física:

$$DirecciónFísica = Tag \parallel Índice \parallel Offset$$

Donde:

- $Bloque = \frac{DirecciónFísica}{TamañodeBloque}$
- $Índice = Bloque$

2.2. Instrumentación de los Algoritmos

Cada acceso a posición de memoria del arreglo $A[i]$ mediante punteros base se traduce en el cálculo explícito de su dirección física relativa para neutralizar la aleatorización del sistema (ASLR):

$$Dirección = BasePtr + (i \times sizeof(int))$$

Los algoritmos evaluados fueron:

- **QuickSort:** Implementación basada en particionamiento de Hoare/Lomuto con llamadas recursivas e intercambio de elementos en sitio (in-place).
- **MergeSort:** Implementación mediante división y conquista, utilizando arreglos auxiliares temporales durante la fase de mezcla (merge).

3. Análisis y Comparación de Resultados

A continuación se consolidan los resultados experimentales obtenidos tras ejecutar la simulación estabilizada sobre el dataset de 10,000 elementos:

Resultados experimentales de simulación de caché (32 KB direct-mapped).

Algoritmo	Bloque (B)	Total Accesos	Aciertos	Fallos	Hit Rate (%)
QuickSort	16	416207	412802	3405	99.18 %
QuickSort	32	416207	414504	1703	99.59 %
QuickSort	64	416207	415352	855	99.80 %
QuickSort	128	416207	415779	428	99.90 %
QuickSort	256	416207	415991	216	99.95 %
MergeSort	16	654943	631482	23461	98.39 %
MergeSort	32	654943	642819	12124	99.19 %
MergeSort	64	654943	648030	6913	99.60 %
MergeSort	128	654943	649902	5041	99.80 %
MergeSort	256	654943	649236	5707	99.90 %

3.1. Discusión de los Resultados

1. **Efecto del Tamaño de Bloque:** Al incrementar el bloque de 16B a 256B, la tasa de aciertos (Hit Rate) aumenta sustancialmente. Esto ocurre porque un bloque de 256 Bytes trae consigo 64 enteros de forma contigua a la caché. Por ende, tras un primer fallo obligatorio (Compulsory Miss), las siguientes 63 lecturas consecutivas resultan en aciertos automáticos debido a la **localidad espacial**.
2. **Comparación QuickSort vs. MergeSort:** MergeSort realiza un patrón de acceso estrictamente secuencial al recorrer los subarreglos L y R durante la mezcla, maximizando el aprovechamiento del ancho de la línea de caché. Por su parte, QuickSort presenta accesos distantes al comparar elementos respecto al pivote desde los extremos del arreglo. Además, MergeSort experimenta un volumen absoluto de accesos a memoria y fallos notablemente mayor debido a la sobrecarga de escritura y lectura hacia sus arreglos auxiliares temporales, penalizando la eficiencia energética frente a la estrategia in-place de QuickSort.

4. Propuesta de Mejora al Recurso de Ayuda

Basado en la experiencia de simulación y el análisis de la documentación de apoyo del proyecto, se plantean las siguientes propuestas generales de mejora pedagógica:

- **Generación de Diagramas Visuales**
  - **Diapositiva 1 (Comparativa de Memoria):** Realiza una diapositiva de presentación profesional y moderna en pantalla ancha titulada "QuickSort vs MergeSort: Impacto en la Memoria" sobre un tema de interfaz de usuario de tecnología oscura. Dos columnas principales: Columna izquierda titulada "QuickSort (In-Place Caché)" con viñetas: Reorganización in-place sin estructuras extras", Çarga contigua de 64B a L1", "Pivote guardado en registros CPU". Columna derecha titulada "MergeSort (Sobrecarga de Memoria)" con viñetas: Requiere memoria auxiliar O(N)", "Fallos de conflicto por direccionamiento disperso", "Tráfico continuo que satura la DRAM". En la parte inferior, un cuadro de color cian brillante titulado "La Pared de la Memoria" que muestra "DRAM 200 ciclos VS Caché L1 ¡4 ciclos". Alta resolución, estilo de diagrama de ingeniería de software limpio, fondo de pizarra oscuro, tipografía nítida, relación de aspecto 16:9.

  - **Diapositiva 2 (Optimización C++):** Una diapositiva de presentación técnica, moderna y nítida, titulada .Alineación a 64B y optimización de pila en C++" sobre un fondo de interfaz oscura y futurista. Diseño dividido: a la izquierda, una ventana de editor de código que muestra la estructura de C++ 'struct alignas(64) CachedNode double val; int id; char pad[52]; ;' con resaltado de sintaxis claro y anotaciones que señalan la alineación de memoria (8 bytes, 4 bytes, relleno de 64B). A la derecha, una lista con viñetas: Ëmpaquetado de memoria: ordenar tipos de mayor a menor tamaño", Älmacenamiento en caché de fusión: pool de búferes preasignados (+3 % a +5 %)", C̈ontrol de pila: definir d̈epth limit para evitar el desbordamiento de pila (Stack Overflow)". En la parte inferior, una etiqueta destacada: "Mitigación de latencia: alineación a 64B frente a

pila controlada". Diseño técnico minimalista y elegante; relación de aspecto 16:9.

- **Inclusión de un Diagrama de Descomposición de Direcciones:** Incorporar esquemas visuales que muestren explícitamente la división de bits (Tag, Index, Offset) para cada tamaño de bloque evaluado (16B hasta 256B), facilitando la comprensión matemática del mapeo.
- **Análisis de Penalización por Fallo (Miss Penalty):** Aclarar conceptualmente que, aunque aumentar el bloque reduce los fallos, en hardware real un bloque más grande requiere mayor tiempo de transferencia desde la memoria principal.
- **Visualización de Patrones de Acceso:** Añadir representaciones gráficas paso a paso del comportamiento de la memoria durante las fases críticas de los algoritmos (el particionamiento en QuickSort vs. la mezcla en MergeSort).